

PRAKTIKUM 3

EVALUASI SISTEM TEMU KEMBALI

Mata Kuliah Sistem Temu Kembali Informasi · Pertemuan 3 · Semester 5

Program Studi	Teknik Komputer
Alokasi Waktu	30 menit (sesi akhir Pertemuan 3)
Perangkat	Google Colab (peramban, tanpa instalasi)
Nama Mahasiswa	Siti Nurkhalisah
NIM / Kelas	240210501046/Sistem Temu Kembali B
Tanggal Praktikum	2 September 2026

A. Daftar Judul Skripsi Yang Digunakan

- 1) Analisis Kerentanan Website E-Skripsi JTIK dan Rancangan Solusi Keamanan Berdasarkan OWASP WSTG
- 2) Implementasi Machine Learning untuk Klasifikasi Sentimen dan Pemodelan Topik Komentar Pengguna pada Video Ulasan Produk Gadget Berbasis Website
- 3) Implementasi Sistem Sinkronisasi Basis Data dengan Metode Redirect untuk Mitigasi Serangan DDoS
- 4) Pengaruh ChatGPT Dependency Terhadap Kemampuan Literasi Digital Mahasiswa Jurusan Teknik Informatika dan Komputer
- 5) Simulasi Sistem Keamanan Jaringan IDS (Intrusion Detection System) Menggunakan Snort pada Jaringan Virtual
- 6) Analisis Kualitas Layanan Jaringan Wireless LAN di Lingkungan Jurusan Teknik Informatika dan Komputer Menggunakan Parameter QoS
- 7) Rancang Bangun Sistem Monitoring Kualitas Air Secara Real-Time Berbasis Internet of Things (IoT) untuk Budidaya Ikan dengan Antarmuka Blynk
- 8) Smart Home Monitoring Pagar Rumah Menggunakan Pengenalan Wajah Berbasis IoT
- 9) Analisis Penerimaan AI ChatGPT dalam Pembelajaran Digital Menggunakan Pendekatan Technology Acceptance Model
- 10) Pengaruh Penggunaan AI Generatif Model UTAUT Terhadap Penyelesaian Tugas Akademik Mahasiswa Prodi PTIK UNM

B. Alat dan Bahan

1. Laptop atau komputer dengan sambungan internet.
2. Akun Google untuk mengakses Google Colab.
3. Tautan notebook praktikum yang dibagikan dosen melalui grup kelas.

C. Alur Praktikum

Kode program:

```
import re
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from Sastrawi.StopWordRemover.StopWordRemoverFactory import StopWordRemoverFactory

documents = [
    "Analisis Kerentanan Website E-Skripsi JTIK dan Rancangan Solusi Keamanan Berdasarkan OWASP WSTG",
    "Implementasi Machine Learning untuk Klasifikasi Sentimen dan Pemodelan Topik Komentar Pengguna pada Video Ulasan Produk Gadget Berbasis Website",
    "Implementasi Sistem Sinkronisasi Basis Data dengan Metode Redirect untuk Mitigasi Serangan DDoS",
    "Pengaruh ChatGPT Dependency Terhadap Kemampuan Literasi Digital Mahasiswa Jurusan Teknik Informatika dan Komputer",
    "Simulasi Sistem Keamanan Jaringan IDS (Intrusion Detection System) Menggunakan Snort pada Jaringan Virtual",
    "Analisis Kualitas Layanan Jaringan Wireless LAN di Lingkungan Jurusan Teknik Informatika dan Komputer Menggunakan Parameter QoS",
    "Rancang Bangun Sistem Monitoring Kualitas Air Secara Real-Time Berbasis Internet of Things (IoT) untuk Budidaya Ikan dengan Antar Muka Blynk",
    "Smart Home Monitoring Pagar Rumah Menggunakan Pengenalan Wajah Berbasis IoT",
    "Analisis Penerimaan AI ChatGPT dalam Pembelajaran Digital Menggunakan Pendekatan Technology Acceptance Model",
    "Pengaruh Penggunaan AI Generatif Model UTAUT Terhadap Penyelesaian Tugas Akademik Mahasiswa Prodi PTIK UNM"
]

doc_ids = [f"D{i+1}" for i in range(len(documents))]

stemmer = StemmerFactory().create_stemmer()
stopword = StopWordRemoverFactory().create_stop_word_remover()

def preprocess(text):
    text = re.sub(r"[^a-z0-9\s]", " ", text.lower())
    text = re.sub(r"\s+", " ", text).strip()
    return stemmer.stem(stopword.remove(text))

processed = [preprocess(doc) for doc in documents]

vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(processed)
```

```
def search(query, top_n=5):
    q_vector = vectorizer.transform([preprocess(query)])
    scores = cosine_similarity(q_vector, tfidf).flatten()

    hasil = pd.DataFrame({
        "Doc_ID": doc_ids,
        "Judul": documents,
        "Score": scores
    })

    return hasil.sort_values(
        "Score", ascending=False
    ).head(top_n)["Doc_ID"].tolist()

ground_truth = {
    "AI": {"D4", "D9", "D10"},
    "Jaringan": {"D5", "D6"},
    "IoT": {"D7", "D8"}
}

def precision(hasil, relevan):
    return sum(d in relevan for d in hasil) / 5

def recall(hasil, relevan):
    return sum(d in relevan for d in hasil) / len(relevan)

def f1(p, r):
    return 0 if p + r == 0 else 2 * p * r / (p + r)

def ap(hasil, relevan):
    hit = 0
    total = 0

    for i, d in enumerate(hasil, 1):
        if d in relevan:
            hit += 1
```

```

total += hit / i

return total / len(relevan) if relevan else 0

hasil = []

for query, relevan in ground_truth.items():
    ranking = search(query)

    p = precision(ranking, relevan)
    r = recall(ranking, relevan)
    f = f1(p, r)
    a = ap(ranking, relevan)

    hasil.append([query, p, r, f, a])

evaluasi = pd.DataFrame(
    hasil,
    columns=["Query", "Precision@5", "Recall@5", "F1", "AP"]
).round(4)

MAP = evaluasi["AP"].mean()

terbaik = evaluasi.loc[evaluasi["AP"].idxmax()]
terburuk = evaluasi.loc[evaluasi["AP"].idxmin()]

print(evaluasi.to_string(index=False))
print("\nMAP:", round(MAP, 4))
print("Query Terbaik:", terbaik["Query"])
print("Query Terburuk:", terburuk["Query"])

```

Output:

```

...   Query  Precision@5  Recall@5   F1   AP
      AI           0.4    0.6667  0.5000  0.6667
  Jaringan           0.4    1.0000  0.5714  1.0000
      IoT           0.4    1.0000  0.5714  1.0000

MAP: 0.8889
Query Terbaik: Jaringan
Query Terburuk: AI

```

D. Analisis

Query yang memiliki performa terburuk adalah “AI” karena memperoleh nilai F1 sebesar 0,5000 dan AP sebesar 0,6667, lebih rendah dibandingkan query “Jaringan” dan “IoT”. Nilai Recall@5 sebesar 0,6667 menunjukkan bahwa dari tiga dokumen yang dianggap relevan, hanya dua dokumen yang berhasil ditemukan dalam lima hasil teratas. Hal ini menunjukkan bahwa mesin pencari belum mampu menemukan seluruh dokumen yang relevan dengan query “AI”. Penyebabnya adalah pencarian berbasis TF-IDF dan cosine similarity sangat bergantung pada kesamaan kata yang muncul dalam judul, sehingga dokumen yang membahas AI secara tidak langsung seperti D4 dapat memperoleh skor lebih rendah. Oleh karena itu, query “AI” menjadi query terburuk karena menghasilkan ketepatan dan kelengkapan pencarian yang lebih rendah dibandingkan query “Jaringan” dan “IoT”.